14 · C 비교 (int 32-bit, 2의 보수): true/false?

Lec2 · 비트·정수 | Data Lab 연계 · 난이도 ★★

문제 14

(1) -1 < 0U (2) (unsigned char)300 == 44 (3) (short)0x12348000의 값 (4) INT_MIN == -INT_MIN (5) unsigned u = 3; int i = -4; 일 때 u + i > 0

(unsigned char)258은 8비트로 잘려 2 · 32-bit에서 (unsigned)-1은 4294967295.

힌트 보기

한쪽이 unsigned면 다른 쪽도 unsigned로 변환 (일반 산술 변환). unsigned 정수형으로 변환하면 결과는 해당 타입의 값 범위에 따른 모듈러 연산으로 결정된다. 반면, 값이 표현 범위를 벗어나는 signed 정수형으로의 변환 결과는 C 구현에 따라 달라질 수 있다. 이 문제의 (3)은 GCC/Clang의 Linux x86-64 구현을 가정한다. 부호 있는 정수 연산에서 결과가 해당 타입의 표현 범위를 벗어나면 ISO C에서 동작은 정의되지 않는다(UB). 2의 보수 표현을 사용하더라도 부호 있는 정수 오버플로가 자동으로 순환한다고 보장되지는 않는다. 반면, unsigned 정수 연산은 해당 타입의 비트 폭에 따른 모듈러 연산을 수행한다.

정답 보기

풀이

① (1) false: 0U 때문에 −1이 unsigned로 바뀌어 4294967295 < 0이 됨

② (2) true: 300 mod 256 = 44

③ (3) 0x12348000의 하위 16비트는 0x8000이다. 이 문제에서 가정한 GCC/Clang Linux x86-64 구현에서는 해당 값을 16비트 short로 변환하면 −32768이 된다. 단, 표현 범위를 벗어나는 signed 정수형 변환의 결과를 모든 C 구현에 일반화해서는 안 된다.

④ (4) UB: INT_MIN은 −2147483648이다. 따라서 -INT_MIN의 수학적 결과인 2147483648은 32비트 int로 표현할 수 없다. 이 부호 반전은 signed overflow를 일으켜 ISO C에서 정의되지 않은 동작이 된다. 따라서 이 식의 결과를 true 또는 false로 확정할 수 없다.

⑤ (5) true: u + i는 unsigned 연산, $3 + (2^{32}-4) = 2^{32}-1 > 0$

최종 답 **(1) false (2) true (3) −32768 (해당 GCC/Clang 구현 기준) (4) UB (5) true**